

WhitePages

⚠ **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

I stopped using YellowPages and moved onto WhitePages... but the page they gave me is all blank!

Hints

- There is data encoded somewhere... there might be an online decoder.
-

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `xxd` – View hexadecimal representation of files
 - **CyberChef** – Web-based tool for data transformation
 - **Python** – Custom script for binary decoding
 - `sed` – Stream editor for text manipulation
-

Complete Solution

Step 1: Download and Prepare

Download the file named `whitepages.txt` from the challenge page.

At first glance, the file appears completely blank – but the challenge name “WhitePages” suggests something is hidden in the whitespace.

Step 2: Basic File Inspection

First, verify the file type:

```
file whitepages.txt
```

Expected output:

```
whitepages.txt: Unicode text, UTF-8 text, with very long lines (1296), with no line terminators
```

The file is a UTF-8 text file with one very long line. It's not actually empty – something is there, just invisible.

Step 3: Search for Visible Text

Check if any readable text reveals the flag:

```
strings whitepages.txt | grep -i "picoCTF"
```

Output: (no results)

No visible flag yet.

Step 4: Examine Metadata

Use `exiftool` to inspect the file's metadata:

```
exiftool whitepages.txt
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : whitepages.txt
Directory                   : .
File Size                    : 2.8 kB
File Modification Date/Time  : 2026:05:20 18:46:09+02:00
```

```
...
File Type           : TXT
File Type Extension : txt
MIME Type           : text/plain
MIME Encoding       : utf-8
Byte Order Mark     : No
Newlines            : (none)
Line Count          : 1
Word Count          : 344
```

Key finding: The file has a `Word Count` of **344** – but the file appears blank! This confirms that invisible characters are present.

Step 5: Hexdump Analysis

Let's examine the hexadecimal representation of the file:

```
xxd whitepages.txt | head
```

Output:

```
00000000: e280 83e2 8083 e280 83e2 8083 20e2 8083  ....
00000010: 20e2 8083 e280 8320 2020 e280 83e2 8083  ....
00000020: e280 83e2 8083 e280 8320 20e2 8083 20e2  ....
00000030: 8083 e280 8320 e280 8320 20e2 8083 e280  ....
00000040: 83e2 8083 2020 e280 8320 20e2 8083 2020  ....
00000050: 2020 e280 8320 e280 83e2 8083 e280 83e2  ...
00000060: 8083 2020 e280 8320 e280 8320 e280 8320  ..
00000070: e280 83e2 8083 e280 8320 e280 83e2 8083  ....
00000080: e280 8320 20e2 8083 e280 83e2 8083 e280  ...
00000090: 83e2 8083 20e2 8083 20e2 8083 e280 83e2  ....
```

[illegible]

Step 6: Identify the Hidden Pattern

Looking at the hexdump, we see repeating patterns:

Hexadecimal	Unicode Character	ASCII Name
e2 80 83	U+2003	EM SPACE
20	U+0020	SPACE

- **EM SPACE** (e2 80 83) – a wide invisible space
- **Regular SPACE** (20) – a normal space

Step 7: Decode the Binary Data

We need to convert the sequence of spaces into binary, then to text.

- e2 80 83 (EM SPACE) → binary 0
- 20 (SPACE) → binary 1

Step 8: Method 1 – CyberChef (Recommended)

1. Go to <https://gchq.github.io/CyberChef/>
2. Upload the `whitepages.txt` file
3. Add the following operations in order:

Operation	Settings
To Hex	No additional settings
Find / Replace	Find: e2 80 83 (EM SPACE), Replace: 0
Find / Replace	Find: 20 (SPACE), Replace: 1
From Binary	Delimiter: None (8-bit chunks)

The screenshot shows the CyberChef web interface. The 'Recipe' panel on the left contains the following operations in order:

- To Hex**: No additional settings.
- Find / Replace**: Find: e2 80 83, Replace: 0, REGEX, Global match checked.
- Find / Replace**: Find: 20, Replace: 1, Global match checked.
- From Binary**: Delimiter: None (8-bit chunks).

The 'Output' panel on the right shows the decoded text:

```
picoCTF{SEE PUBLIC RECORDS & BACKGROUND REPORT
5000 Forbes Ave, Pittsburgh, PA 15213
picoCTF{[REDACTED]}
```

The 'File details' panel on the right shows the file 'whitepages.txt' with a size of 2764 bytes and a type of 'unknown'.

The decoded text will appear, containing the flag!

Step 9: Method 2 – Python Script

Here's a Python script to decode the whitespace:

python

```
#!/usr/bin/env python3
"""
WhitePages Decoder - Converts EM SPACE and SPACE to binary and decodes to ASCII.
"""

def decode_whitepages(filename):
    # Read the file
    with open(filename, 'rb') as f:
        data = f.read()

    # Convert bytes to hex string
    hex_str = data.hex()

    # Replace EM SPACE (e28083) with '0' and regular SPACE (20) with '1'
    binary = hex_str.replace('e28083', '0').replace('20', '1')

    # Remove any remaining spaces (should be none)
    binary = binary.replace(' ', '')

    # Split into 8-bit chunks and decode
    result = ''
    for i in range(0, len(binary), 8):
        if i + 8 <= len(binary):
            byte = binary[i:i+8]
            result += chr(int(byte, 2))

    return result

if __name__ == "__main__":
    flag = decode_whitepages("whitepages.txt")
    print(flag)
```

Output:

picoCTF{<REDACTED>}

Step 10: Method 3 – Command Line One-Liner

```
xxd -p whitepages.txt | tr -d '\n' | sed 's/e28083/0/g' | sed 's/20/1/g' | python3 -c
"import sys; b=sys.stdin.read().strip(); print(''.join(chr(int(b[i:i+8],2)) for i in
```

```
range(0, len(b), 8)))"
```

Explanation of the pipeline:

Command	Purpose
<code>xxd -p</code>	Convert file to hex (plain)
<code>tr -d '\n'</code>	Remove newlines
<code>sed 's/e28083/0/g'</code>	Replace EM SPACE with <code>0</code>
<code>sed 's/20/1/g'</code>	Replace SPACE with <code>1</code>
<code>python3 -c "..."</code>	Convert binary to ASCII

Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, decoding the whitespace will reveal the complete flag.

Key Concepts

Concept	Description
Whitespace Steganography	Hiding data using invisible whitespace characters (spaces, tabs, zero-width spaces).
EM SPACE (U+2003)	A wide space character that is invisible but has a distinct Unicode code point.
Binary Encoding	Using two different characters to represent 0 and 1, then grouping into 8-bit bytes.

Concept	Description
xxd	Tool to view and manipulate hexadecimal representations of files.
Unicode	Standard for encoding characters; many invisible characters exist.

Pro Tips

1. **Word Count is a clue** – If a text file appears blank but has a non-zero word count, invisible characters are present.
2. **Look for repeating hex patterns** – In this case, `e28083` and `20` repeating indicates a binary encoding.
3. **EM SPACE is not the only invisible character** – Other common ones include:
 - `e2808b` (U+200B) – Zero-width space
 - `e2808c` (U+200C) – Zero-width non-joiner
 - `e2808d` (U+200D) – Zero-width joiner
4. **CyberChef is excellent for this** – The “Find / Replace” operation can substitute hex patterns, and “From Binary” converts to text.
5. **Test both mappings** – If `0` and `1` don’t work, try swapping them (EM SPACE = 1, SPACE = 0).
6. **The pipeline approach saves time** – Once you have the one-liner, you can reuse it for similar challenges.

Alternative Decoding Tools

Tool	Method
CyberChef	To Hex → Find/Replace (<code>e28083</code> → <code>0</code> , <code>20</code> → <code>1</code>) → From Binary
Python Script	Use the script provided above

Tool	Method
Command Line	<code>xxd -p file \ sed 's/e28083/0/g' \ sed 's/20/1/g' \ python3 -c "..."</code>